



LeetCode 1094: Car Pooling

1. Problem Title & Link

Title: LeetCode 1094: Car Pooling

Link: <https://leetcode.com/problems/car-pooling/>

2. Problem Statement (Short Summary)

A vehicle has a fixed seating capacity.

You are given trips:

[numPassengers, from, to]

Meaning:

- numPassengers get into the car at location from
- They get out at location to

Determine whether it is possible to complete all trips without exceeding the car's capacity at any point.

Return:

true -> possible

false -> capacity exceeded

3. Examples (Input → Output)

Example 1

Input:

trips = [[2,1,5],[3,3,7]]

capacity = 4

Output:

False

Explanation:

At location 3:

2 passengers already inside

+3 new passengers

Total = 5

Capacity = 4

Not possible.

Example 2

Input:

trips = [[2,1,5],[3,3,7]]

capacity = 5



Output:

True

Example 3

Input:

trips = [[3,2,7],[3,7,9],[8,3,9]]

capacity = 11

Output:

True

4. Constraints

- $1 \leq \text{trips.length} \leq 1000$
- $1 \leq \text{passengers} \leq 100$
- $0 \leq \text{from} < \text{to} \leq 1000$
- $1 \leq \text{capacity} \leq 100000$

Important observation:

Locations are only from 0 to 1000

This hints toward Prefix Sum / Difference Array.

5. Core Concept (Pattern / Topic)

Difference Array + Prefix Sum

Related Patterns:

- Sweep Line
- Range Updates
- Interval Processing

6. Thought Process (Step-by-Step Explanation)

Brute Force

For every trip:

Increase passenger count

for every location

between from and to

Example:

[2,1,5]

Location:

1 2 3 4



+2 +2 +2 +2

Complexity:

$O(\text{trips} \times \text{locations})$

Works but not elegant.

Optimized Thinking

Instead of updating all locations:

When passengers enter:

`diff[from] += passengers`

When passengers leave:

`diff[to] -= passengers`

Later compute prefix sums.

The running sum tells us how many passengers are inside the car at every location.

If it exceeds capacity:

`return False`

Otherwise:

`return True`

Why It Works

Difference array efficiently stores:

Passenger enters

Passenger leaves

Prefix sum reconstructs the actual passenger count at each point.

7. Visual / Intuition Diagram (ASCII Diagram)

```
Trip:
[2,1,5]
Difference Array:
Location

0 1 2 3 4 5

0 2 0 0 0 -2
Prefix Sum:
0 2 2 2 2 0
Meaning:
2 passengers from location 1 to 4
```

8. Pseudocode (Language Independent)



```
Create diff array

For each trip:

    diff[from] += passengers

    diff[to] -= passengers

currentPassengers = 0

For each location:

    currentPassengers += diff[location]

    if currentPassengers > capacity:
        return False

return True
```

9. Code Implementation

✓ Python

```
class Solution:
    def carPooling(self, trips, capacity):

        diff = [0] * 1001

        for passengers, start, end in trips:

            diff[start] += passengers
            diff[end] -= passengers

        current = 0

        for passengers in diff:

            current += passengers

            if current > capacity:
                return False

        return True
```

✓ Java

```
class Solution {

    public boolean carPooling(int[][] trips, int capacity) {
```

```
int[] diff = new int[1001];

for (int[] trip : trips) {

    int passengers = trip[0];
    int start = trip[1];
    int end = trip[2];

    diff[start] += passengers;
    diff[end] -= passengers;
}

int current = 0;

for (int value : diff) {

    current += value;

    if (current > capacity)
        return false;
}

return true;
}
```

10. Time & Space Complexity

Time Complexity

$O(n + 1000)$

where:

n = number of trips

Since locations are bounded by 1000.

Effectively:

$O(n)$

Space Complexity

$O(1000)$

Difference array.

Constant space relative to input.

11. Common Mistakes / Edge Cases

Mistake 1

Using:



diff[end - 1]

instead of

diff[end]

Passengers leave at location end.

Mistake 2

Checking capacity before applying prefix sum.

Need actual passenger count.

Mistake 3

Sorting trips unnecessarily.

Not needed.

12. Detailed Dry Run (Step-by-Step Table)

Input:

trips = [[2,1,5],[3,3,7]]

capacity = 4

Difference Array Updates:

Trip 1

diff[1] += 2

diff[5] -= 2

Trip 2

diff[3] += 3

diff[7] -= 3

Important positions:

Location	Difference
1	2
3	3
5	-2
7	-3

Prefix Sum:

Location	Current Passengers
0	0
1	2



2	2
3	5
4	5

At location 3:

$5 > 4$

Return:

False

13. Common Use Cases (Real-Life / Interview)

Ride Sharing Systems

Track passenger count during trips.

Bus Occupancy

Monitor seats occupied between stations.

Airline Boarding Simulation

Passengers enter and leave at checkpoints.

Traffic Analytics

Count vehicles in road segments.

14. Common Traps (Important!)

- Confusing start/end handling
- Forgetting prefix sum
- Using brute force updates
- Updating every location individually
- Not checking capacity at every point

15. Builds To (Related LeetCode Problems)

1. LC 303 — Range Sum Query
2. LC 560 — Subarray Sum Equals K
3. LC 1109 — Corporate Flight Bookings
4. LC 2381 — Shifting Letters II
5. LC 1094 — Car Pooling

These are classic Difference Array problems.

16. Alternate Approaches + Comparison



Approach	Time	Space	Notes
Brute Force	$O(n \times 1000)$	$O(1)$	Slow
Sorting Events	$O(n \log n)$	$O(n)$	Good
Difference Array	$O(n + 1000)$	$O(1000)$	Optimal

17. Why This Solution Works (Short Intuition)

Every trip contributes passengers at the start location and removes them at the end location. The difference array records these changes efficiently, and the prefix sum reconstructs the number of passengers inside the car at every point. If the count ever exceeds capacity, the trip schedule is impossible.

18. Variations / Follow-Up Questions

Follow-Up 1

What if locations go up to:

10^9

Use:

Sweep Line + Sorting Events

instead of a fixed array.

Follow-Up 2

What if trips can be canceled dynamically?

Use:

Fenwick Tree

Segment Tree

Follow-Up 3

What if we need the maximum occupancy?

Keep track of:

$\text{maxPassengers} = \max(\text{maxPassengers}, \text{current})$

during prefix sum traversal.